I File

Il file e` l'unita` logica di memorizzazione dei dati su memoria di massa.

- Consente una ***memorizzazione persistente*** dei dati, ***non limitata*** dalle dimensioni della memoria centrale.

- Generalmente un file e` una sequenza di componenti omogenee (***record logici***).

- I file sono gestiti dal Sistema Operativo sottostante. Per ogni versione C esistono funzioni per il trattamento dei file (***Standard Library***) che tengono conto delle funzionalita` del S.O.

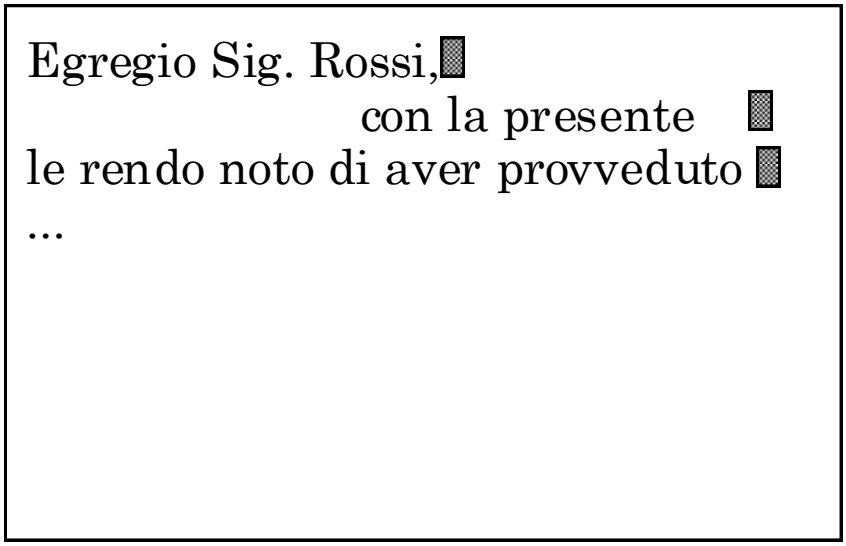
In C i file vengono distinti in due categorie:

- ***file di testo***, costituiti da caratteri ed organizzati in linee (ciascuna terminata da '\n')

- ***file binari***, visti come sequenze di bit

File di testo

Sono file di caratteri, organizzati in linee.
Ogni linea e` terminata da una marca di fine linea (***newline***, carattere '\n').



☞ Il ***record logico*** puo` essere il singolo carattere oppure la singola linea.

Organizzazione degli archivi

Dispositivi di memoria di massa:

- ***Ad accesso sequenziale***, per accedere ad un blocco occorre scandire tutti quelli che lo precedono (ad esempio, nastri).

- ***Ad accesso diretto***, ciascun blocco e` indirizzabile (ad esempio, dischi).

I dati sono organizzati in memoria di massa con l'obiettivo di recuperare in modo efficiente i dati.

Metodi base per l'organizzazione degli archivi (***strutture dati***):

- Sequenziale
- Ad accesso diretto (o casuale)
- Ad indice

A ciascuna struttura dati corrispondono diversi modi nei quali è possibile utilizzarla (ad esempio, diversi ***metodi di elaborazione***).

Gestione di file in C

Sono ***file sequenziali,*** con possibilita` di ***accesso diretto***.

Apertura di un file:

☞ Prima di accedere ad un file e` necessario ***aprirlo:*** l'operazione di apertura compie le azioni preliminari necessarie affinche` il file possa essere acceduto (in lettura o in scrittura)

Per aprire un file in C:

- e` necessario definire una variabile di tipo *puntatore a FILE,* che si dichiara così:

  FILE *fp;

☞ il tipo puntatore a FILE e` un tipo non primitivo dichiarato nel file **stdio.h.**

- si utilizza la specifica **funzione di apertura**:

FILE ***fopen(char *name, char *mode);**

Apertura di un File

FILE *fopen(char ****name**, char ****mode****);**

dove:

- ***name*** e` una stringa che rappresenta il nome (assoluto o relativo) del file nel file system
- ***mode*** esprime la modalita` di accesso scelta.
  - **"r"**, in lettura (read)
  - **"w"**, in scrittura (write)
  - **"a"**, scrittura, aggiunta in fondo (append)
  - **"b"**, a fianco ad una delle precedenti, indica che il file e` binario
  - **"t"**, a fianco ad una delle precedenti, indica che il file e`di testo

L'operazione di apertura ritorna come risultato un ***puntatore*** al file aperto:

FILE *fopen(char *name, char *mode);

Il file deve esistere in memoria di massa. Altrimenti restituisce il valore NULL.

Dipendentemente dalla modalita` di apertura scelta il file puo` essere acceduto con diversi strumenti:

- fprintf
- fscanf
- fread
- fwrite
- etc.

Chiusura di un File

☞ Al termine di una sessione di accesso al file, esso deve essere chiuso.

L'operazione di chiusura si realizza con la funzione **fclose**:

```
int  fclose (FILE     *fp);
```

dove:

- fp rappresenta il puntatore al file da chiudere.

fclose ritorna come risultato un intero:

- Se l'operazione di chiusura e` eseguita correttamente restituisce il valore 0
- se la chiusura non e` andata a buon fine, ritorna la costante EOF.

☞ **fclose** assegna il valore NULL al puntatore fp.

fp rappresenta, dall'apertura in poi, il riferimento da utilizzare nelle operazioni di accesso a c:\anna\dati. Esso individua, in particolare:

- il file
- l'elemento corrente all'interno del file

Esempio:

```
#include <stdio.h>
main()
{
  FILE *fp;
  fp = fopen("prova.dat", "w")
  ...
  fclose(fp);
  return 0;
}
```

File standard di I/O

Esistono tre file testo che sono aperti automaticamente all'inizio di ogni esecuzione:

- ***stdin***, standard input (tastiera), aperto in lettura
- ***stdout***, standard output (video), aperto in scrittura
- ***stderr***, standard error (video), aperto in scrittura

☞ *stdin*, *stdout*, *stderr* sono variabili di tipo *puntatore a file* automaticamente (ed **implicitamente**) definite
› **non vanno definite**.

Funzione feof()

Durante la fase di accesso ad un file e` possibile verificare la presenza della marca di fine file con la funzione di libreria:

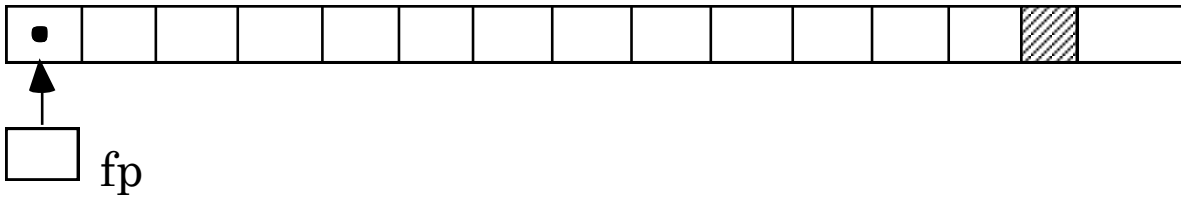
```
int feof(FILE *fp);
```

- ☞ feof(fp) controlla se e` stata raggiunta la fine del file fp nella operazione di lettura o scrittura precedente. Restituisce il valore 0 (falso logico) se non e` stata raggiunta la fine del file, altrimenti un valore diverso da zero (vero logico).

Apertura in lettura

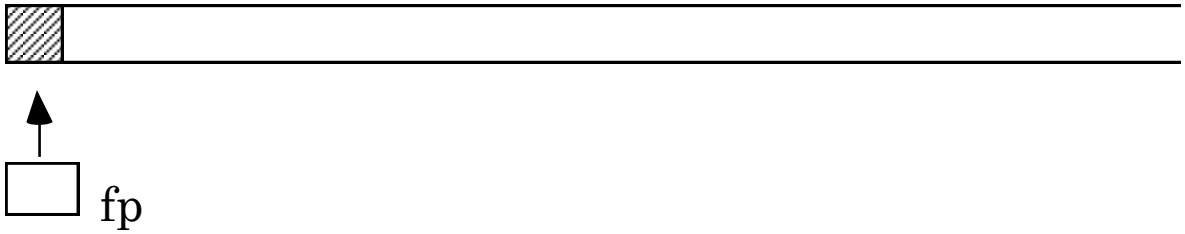
fp = fopen("filename", "r")

- **Se il file non e` vuoto:**



feof(fp) ha valore falso (valore zero).

- **Se il file e` vuoto:**

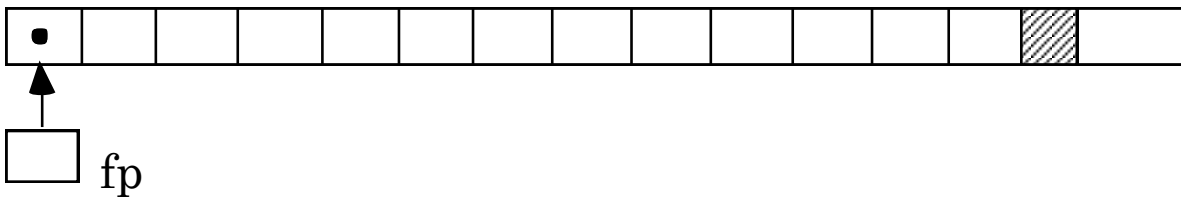


feof(fp) ha valore vero (valore diverso da zero).

Apertura in scrittura

fp = fopen("filename", "w")

- **Anche se il file non e` vuoto:**

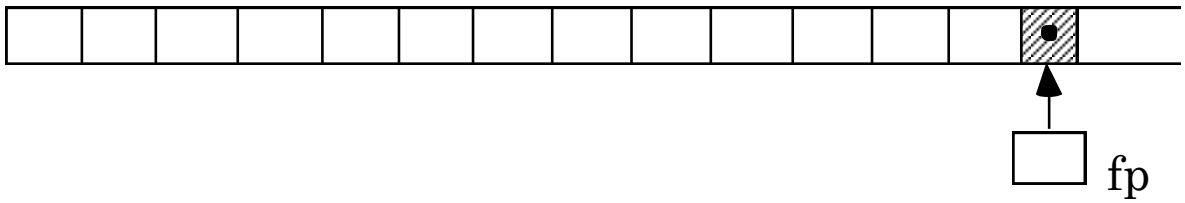


› Se il file esisteva gia`, il suo contenuto viene **perso**.

☞ **feof**(f) ha sempre valore vero.

Apertura in aggiunta (append)

fp = fopen("filename", "a")



Il puntatore al file si posiziona sull'elemento successivo all'ultimo significativo del file › se il file esisteva gia`, il suo contenuto non viene perso.

Lettura e Scrittura di file

Una volta aperto un file, su di esso si puo` accedere in lettura e/o scrittura, compatibilmente con quanto specificato in fase di apertura.

Quattro diverse modalita`:

- Lettura/scrittura **con formato**
- Lettura/scrittura di **caratteri**
- Lettura/scrittura di **stringhe di caratteri**
- Lettura/scrittura di **blocchi** (file **binari**)

Lettura/scrittura con formato:

Funzioni simili a **scanf** e **printf**, ma con un parametro aggiuntivo rappresentante il puntatore al file (di testo) sul quale si vuole leggere o scrivere:

int fscanf **(FILE *fp, *stringa-controllo*, *ind-elem*);**

int fprintf **(FILE *fp, *stringa-controllo*, *elementi*);**

Restituiscono il numero di elementi scritti o letti, oppure un valore negativo in caso di errore.

☞ **notare** che:

scanf **(stringa-controllo, ind-elementi);**
printf **(stringa-controllo, elementi)**

equivalgono a:

fscanf **(stdin, stringa-controllo, ind-elementi);**
fprintf **(stout, stringa-controllo, elementi);**

Esempio:

```
#include <stdio.h>

void riempifile(char *nome,char *str,
int n);
void copiafile(char *nome1,char *nome2);
void vedifile(char *nome);

main()
{
char buf[20]="ciao!";

riempifile("Pippo", buf, 2);
vedifile("Pippo");
copiafile("Pippo", "Pluto");
vedifile("Pluto");
}

void riempifile(char *nome,char *str,
int nvolte)
{
FILE *fp;
int i;
fp=fopen(nome, "w");
for (i=0; i<nvolte; i++)
    fprintf(fp,"%s\n", str);
fclose(fp);
}
```

```
void copiafile(char *nome1,char *nome2)
{
FILE *fp1, *fp2;
char C;

fp1=fopen(nome1, "r");
fp2=fopen(nome2, "w");

while (!feof(fp1))
  {
   fscanf(fp1,"%c", &C);
   fprintf(fp2, "%c", C);
  }

fclose(fp1);
fclose(fp2);
}


void vedifile(char *nome)
{ FILE *fp;
char C;
fprintf(stdout, "contenuto di %s:\n", nome);
fp=fopen(nome, "r");
while (!feof(fp))
  {
   fscanf(fp,"%c", &C);
   fprintf(stdout, "%c", C);
  }
fclose(fp);
}
```

Lettura/scrittura di caratteri:

Funzioni simili a **getchar** e **putchar**, ma con un parametro aggiuntivo rappresentante il puntatore al file (di testo) sul quale si vuole leggere o scrivere:

```
int   getc (FILE *fp);

int   putc (int c, FILE *fp);

int   fgetc (FILE *fp);

int   fputc (int c, FILE *fp);
```

☞ In caso di esecuzione corretta, restituiscono il carattere letto o scritto come intero, altrimenti EOF.

Esempio:

Programma che concatena i file dati come argomento in un unico file (***stdout***)

```
#include <stdio.h>

main(int argc, char **argv)
{
    FILE *fp;
    void filecopy(FILE *, FILE *);

    if (argc==1) filecopy(stdin, stdout);
    else
      while (--argc>0)
        if ((fp=fopen(*++argv, "r"))==NULL)
         {
          printf("\nImpossibile aprire il
               file %s\n", *argv);
          exit(1);
         }
        else
          {filecopy(fp, stdout);
           fclose(fp);
          }
    return 0;
}

void filecopy(FILE *inputFile,
                      FILE *outputFile)
{int c;

    while((c=getc(inputFile))!=EOF)
     putc(c, outputFile);
}
```

Note sull'esempio:

- se non ci sono argomenti (argc=1), il programma copia lo standard input nello standard output;
- la funzione **filecopy** effettua la copia del file carattere per carattere;
- se uno dei file indicati come argomento non esiste, la funzione **fopen** fallisce, restituendo il valore NULL. In questo caso il programma termina (**exit**) restituendo il valore 1 e stampando un messaggio di errore;
- sarebbe meglio scrivere i messaggi di errore sullo standard error (***ridirezione***).

```
fprintf(stderr,"\nImpossibile aprire
       il file %s\n", *argv);
```

Per non perdere dati, la funzione **exit** chiude automaticamente ogni file aperto.

- il ciclo:

```
while((c=getc(inputFile))!=EOF)
    putc(c, outputFile);
```

  poteva essere scritto anche come:

```
while(!feof(inputFile))
 { c=getc(inputFile);
   putc(c, outputFile);}
```

Lettura/scrittura di stringhe

Funzioni simili a **gets** e **puts**:

char*fgets (char *s, int n, FILE *fp);

Trasferisce nella stringa s i caratteri letti dal file puntato da fp, fino a quando ha letto n-1 caratteri, oppure ha incontrato un newline, oppure la fine del file. La **fgets** mantiene il newline nella stringa s.

Restituisce la stringa letta in caso di corretta terminazione; '\0' in caso di errore o fine del file.

int *fputs (char *s, FILE *fp);

Trasferisce la stringa s (terminata da NULL) nel file puntato da fp. Non copia il carattere terminatore NULL ne` aggiunge un newline finale.

Restituisce l'ultimo carattere scritto in caso di terminazione corretta; EOF altrimenti.

Lettura/scrittura di blocchi

Si puo` leggere o scrivere da un file (binario) un intero blocco di dati (binari).

Un file binario memorizza dati di qualunque tipo, in particolare dati che non sono caratteri (interi, reali, vettori o strutture).

Il test di fine del file viene ottenuto con la funzione feof (poiché avendo letto un blocco non si può confrontare un carattere letto ed EOF).

```
int    feof (FILE *fp);
```

☞ feof(fp) controlla se e` stata raggiunta la fine del file fp nella operazione di lettura o scrittura precedente. Restituisce il valore 0 se non e` stata raggiunta la fine del file, altrimenti un valore diverso da zero.

NON VEDIAMO IN DETTAGLIO LA LETTURA E SCRITTURA A BLOCCHI